

OBJECT ORIENTED PROGRAMMING

Unit-V

Java Beans - Application Builder Tools - Using the Bean Developer Kit -Jar Files-Introspection – BDK - Using Bean Info Interface – Persistence - Java Beans API Using Bean Builder - Networking Basics - Java and the Net – InetAddress – TCP/IP Client Sockets – URL – URL Connection – TCP/IP Server - Sockets - A Caching Proxy HTTP Server – Datagram's – RMI.

PART-A

2-Marks

1. What is Serialization?

Serialization in Java is a mechanism of *writing the state of an object into a byte-stream*. The serialization process is platform-independent. Serialization in Java allows us to convert an Object to stream that we can send over the network or save it as file or store in DB for later usage.

2. What is java JNI?

Java Native Interface (JNI) is a native programming interface. It allows Java code that runs inside a Java Virtual Machine (VM) to interoperate with applications and libraries written in other programming languages. An interface that allows Java to interact with code written in another language.

3. What is meant by bean customization?

Customization provides a means for modifying the appearance and behavior of a bean within an application builder so it meets your specific needs.

4. List any two protocols used in RMI

- HTTP POST Protocol
- RMI Transport Protocol
- The Object Serialization protocol

The RMI protocol makes use of two other protocols for its on-the-wire format: Java Object Serialization and HTTP. The Object Serialization protocol is used to marshal call and return data. The HTTP protocol is used to "POST" a remote method invocation and obtain return data when circumstances warrant. Each protocol is documented as a separate grammar.

5. What is the use of java RMI?

- Handles threads and Sockets for underlying communication.
- Server-side implementation can be changed without the knowledge of the client side.
- RMI is multi-threaded, allowing your servers to exploit Java threads for better concurrent processing of client requests. RMI uses built-in Java

security mechanisms that allow your system to be safe when users downloading implementations.

- RMI uses the security manager defined to protect systems from hostile applets to protect your systems and network from potentially hostile downloaded code

6. What is meant by Socket Programming in Java?

Socket programming is a way of connecting two nodes on a network to communicate with each other. One **socket** (node) listens on a particular port at an IP, while other *socket* reaches out to the other in order to form a connection. Java Socket programming is used for communication between the applications running on different JRE.

7. What is socket?

A **socket** in Java is one endpoint of a two-way communication link between two programs running on the network. A **socket** is bound to a port number so that the TCP layer can identify the application that data is destined to be sent to.

8. What types of application prefer UDP rather than TCP?

UDP is faster, simpler, and more efficient than TCP because TCP is connection-oriented and UDP is connectionless. TCP is reliable and it guarantees the delivery of data to the destination router. UDP is not guarantees the delivery of data to the destination router.

9. How to set-up TCP connection.

TCP is connection-oriented. A connection-oriented transport protocol establishes a virtual path between the source and destination. Using a single virtual pathway for the entire message facilitates the acknowledgment process as well as retransmission of damaged or lost frames.

In TCP, connection-oriented transmission requires three phases:

1. Connection establishment
2. Data transfer
3. Connection termination.

10. What is an InetAddress?

Java InetAddress class represents an IP address. The `java.net.InetAddress` class provides methods to get the IP of any host name. An IP address is represented by 32-bit or 128-bit unsigned number.

for example `www.javatpoint.com`, `www.google.com`, `www.facebook.com`, etc.

11. How to create a TCP/IP Socket?

Java [TCP/IP](#) server and client socket applications are created. One for sending character data over a TCP/IP socket connection and another one for sending binary data –*an image is used*–, over a TCP/IP socket connection.

The client assumes that the server is listening for connection request on port **serverPort** via TCP.

```
int serverPort = 4020;
InetAddress host = InetAddress.getByName("localhost");
Socket socket = new Socket(host,serverPort);
```

The server begins by creating a **ServerSocket**:

```
int serverPort = 4020;
ServerSocket serverSocket = new ServerSocket(serverPort);
```

12. Define persistence.

The Java Persistence API (JPA) is a specification of Java. It is used to persist data between Java object and relational database. JPA acts as a bridge between object-oriented domain models and relational database systems. it doesn't perform any operation by itself. It requires an implementation. So, ORM tools like Hibernate, TopLink and iBatis implements JPA specifications for data persistence.

13. What is a bean persistence property?

A bean has the property of persistence when its properties, fields, and state information are saved to and retrieved from storage. The mechanism that makes persistence possible is called *serialization*. Object serialization means converting an object into a data stream and writing it to storage.

14. What are properties of java bean class?

A JavaBean property is a named feature that can be accessed by the user of the object. The feature can be of any Java data type, containing the classes that you define. A JavaBean property may be read, write, read-only, or write-only. JavaBean features are accessed through two methods in the JavaBean's implementation class:

1. getPropertyNames ()

For example, if the property name is firstName, the method name would be getFirstName() to read that property. This method is called the accessor.

2. **setPropertyName ()**

For example, if the property name is firstName, the method name would be setFirstName() to write that property. This method is called the mutator.

15. What is RMI?

The **RMI** (Remote Method Invocation) is an API that provides a mechanism to create distributed application in java. The RMI allows an object to invoke methods on an object running in another JVM.

16. Define Datagram.

A datagram is an independent, self-contained message sent over the network whose arrival, arrival time, and content are not guaranteed. Datagrams play a vital role as an alternative. Datagrams are bundles of information passed between machines. Once the datagram has been released to its intended target, there is no assurance that it will arrive or even that someone will be there to catch it.

17. List out any two advantages of Java Beans.

- The properties, events, and methods of a Bean that are exposed to an application builder tool can be controlled.
- A Bean may be designed to operate correctly in different locales, which makes it useful in global markets.
- Auxiliary software can be provided to help a person configure a Bean. This software is only needed when the design-time parameters for that component are being set. It does not need to be included in the run-time environment.
- A Bean may register to receive events from other objects and can generate events that are sent to other objects.

18. Explain the term Utility Packages.

It contains the collections framework, legacy collection classes, event model, date and time facilities, internationalization, and miscellaneous utility classes (a string tokenizer, a random-number generator, and a bit array).

19. Define API

An API can be described as a way to enable computers to possess a common interface, to allow them to communicate with each other. Java Application Programming Interface (API) is the area of Java development kit (JDK). An API includes classes, interfaces, packages and also their methods, fields, and constructors.

PART-B

11-Marks**20. Discuss about detail in RMI.**

The **RMI** (Remote Method Invocation) is an API that provides a mechanism to create a distributed application in **Java**. The RMI allows an object to invoke methods on an object running in another **JVM**. The Remote Method Invocation provides remote communication between the applications using two objects *stub* and *skeleton*.

A **remote object** is an object whose method can be invoked from another JVM. Let's understand the stub and skeleton objects:

stub

The stub is an object, acts as a gateway for the client side. All the outgoing requests are routed through it. It resides at the client side and represents the remote object. When the caller invokes method on the stub object, it does the following tasks:

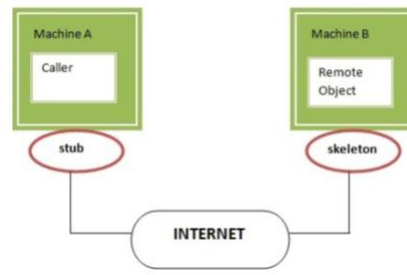
1. It initiates a connection with remote Virtual Machine (JVM),
2. It writes and transmits (marshals) the parameters to the remote Virtual Machine (JVM),
3. It waits for the result
4. It reads (unmarshals) the return value or exception, and
5. It finally, returns the value to the caller.

skeleton

The skeleton is an object, acts as a gateway for the server side object. All the incoming requests are routed through it. When the skeleton receives the incoming request, it does the following tasks:

1. It reads the parameter for the remote method
2. It invokes the method on the actual remote object, and
3. It writes and transmits (marshals) the result to the caller.

In the Java 2 SDK, an stub protocol was introduced that eliminates the need for skeletons.



Steps to implement Interface

1. Defining a remote interface
2. Implementing the remote interface
3. Creating Stub and Skeleton objects from the implementation class using rmic (rmi compiler)
4. Start the rmiregistry
5. Create and execute the server application program
6. Create and execute the client application program.

Step1: Defining the remote interface

The first thing to do is to create an interface which will provide the description of the methods that can be invoked by remote clients. This interface should extend the Remote interface and the method prototype within the interface should throw the RemoteException.

// Creating a Search interface

```
import java.rmi.*;
public interface Search extends Remote
{
    // Declaring the method prototype
    public String query(String search) throws RemoteException;
}
```

Step 2: Implementing the remote interface

The next step is to implement the remote interface. To implement the remote interface, the class should extend to UnicastRemoteObject class of java.rmi package. Also, a default constructor needs to be created to throw the java.rmi.RemoteException from its parent constructor in class.

```
// Java program to implement the Search interface
import java.rmi.*;
import java.rmi.server.*;
public class SearchQuery extends UnicastRemoteObject
    implements Search
{
    // Default constructor to throw RemoteException
    // from its parent constructor
    SearchQuery() throws RemoteException
    {
        super();
    }

    // Implementation of the query interface
    public String query(String search)
```

```
        throws RemoteException
    {
        String result;
        if (search.equals("Reflection in Java"))
            result = "Found";
        else
            result = "Not Found";

        return result;
    }
}
```

Step 3: Creating Stub and Skeleton objects from the implementation class using rmic

The rmic tool is used to invoke the rmi compiler that creates the Stub and Skeleton objects. Its `prototype` is `rmic classname`. For above program the following command need to be executed at the command prompt

```
rmic SearchQuery
```

STEP 4: Start the rmiregistry
Start the registry service by issuing the following command at the command prompt `start rmiregistry`

STEP 5: Create and execute the server application program
The next step is to create the server application program and execute it on a separate command prompt.

- The server program uses `createRegistry` method of `LocateRegistry` class to create `rmiregistry` within the server JVM with the port number passed as argument.
- The `rebind` method of `Naming` class is used to bind the remote object to the new name.

```
//program for server application
```

```
import java.rmi.*;
```

```
import java.rmi.registry.*;
```

```
public class SearchServer
```

```
{
```

```
    public static void main(String args[])
```

```
{
```

```
try

{

    // Create an object of the interface

    // implementation class

    Search obj = new SearchQuery();


    // rmiregistry within the server JVM with

    // port number 1900

    LocateRegistry.createRegistry(1900);


    // Binds the remote object by the name

    // geeksforgeeks

    Naming.rebind("rmi://localhost:1900"+

                  "/geeksforgeeks",obj);

}

catch(Exception ae)

{

    System.out.println(ae);

}
```



```

    }

}

}

```

STEP 4: Start the rmiregistry
Start the registry service by issuing the following command at the command prompt start rmiregistry

STEP 5: Create and execute the server application program
The next step is to create the server application program and execute it on a separate command prompt.

- The server program uses createRegistry method of LocateRegistry class to create rmiregistry within the server JVM with the port number passed as argument.
- The rebind method of Naming class is used to bind the remote object to the new name.

Step 6: Create and execute the client application program
The last step is to create the client application program and execute it on a separate command prompt .The lookup method of Naming class is used to get the reference of the Stub object.

```

//program for client application
import java.rmi.*;
public class ClientRequest
{
    public static void main(String args[])
    {
        String answer,value="Reflection in Java";
        try
        {
            // lookup method to find reference of remote object
            Search access =
                (Search)Naming.lookup("rmi://localhost:1900"+
                                    "/geeksforgeeks");
            answer = access.query(value);
            System.out.println("Article on " + value +
                               " " + answer+" at GeeksforGeeks");
        }
        catch(Exception ae)
        {
            System.out.println(ae);
        }
    }
}

```

Note: The above client and server program is executed on the same machine so localhost is used. In order to access the remote object from another machine, localhost is to be replaced with the IP address where the remote object is present.

21. Explain the collection classes in java.

The **Collection in Java** is a framework that provides an architecture to store and manipulate the group of objects.

Java Collections can achieve all the operations that you perform on a data such as searching, sorting, insertion, manipulation, and deletion

Java Collection means a single unit of objects. Java Collection framework provides many interfaces (Set, List, Queue, Deque) and classes (ArrayList, Vector, LinkedList, PriorityQueue, HashSet, LinkedHashSet, TreeSet).

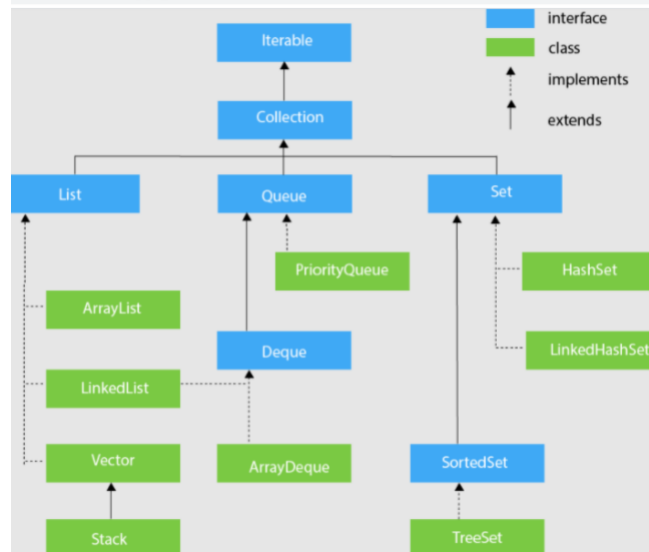
Framework in Java

- It provides readymade architecture.
- It represents a set of classes and interfaces.
- It is optional.

Collection framework

The Collection framework represents a unified architecture for storing and manipulating a group of objects. It has:

1. Interfaces and its implementations, i.e., classes
2. Algorithm



Class: A class is a user-defined blueprint or prototype from which objects are created. It represents the set of properties or methods that are common to all objects of one type.

Interface: Like a class, an interface can have methods and variables, but the methods declared in an interface are by default abstract (only method signature, no body). Interfaces specify what a class must do and not how. It is the blueprint of the class.

Interfaces which extend the Collections Interface

The collection framework contains multiple interfaces where every interface is used to store a specific type of data. The following are the interfaces present in the framework.

1. Iterable Interface: This is the root interface for the entire collection framework. The collection interface extends the iterable interface. Therefore, inherently, all the interfaces and classes implement this interface. The main functionality of this interface is to provide an iterator for the collections. Therefore, this interface contains only one abstract method which is the iterator. It returns the *Iterator iterator();*

2. Collection Interface: This interface extends the iterable interface and is implemented by all the classes in the collection framework. This interface contains all the basic methods which every collection has like adding the data into the collection, removing the data, clearing the data, etc. All these methods are implemented in this interface because these methods are implemented by all the classes irrespective of their style of implementation. And also, having these methods in this interface ensures that the names of the methods are universal for all the collections. Therefore, in short, we can say that this interface builds a foundation on which the collection classes are implemented.

3. List Interface: This is a child interface of the collection interface. This interface is dedicated to the data of the list type in which we can store all the ordered collection of the objects. This also allows duplicate data to be present in it. This list interface is implemented by various classes like ArrayList, Vector, Stack, etc. Since all the subclasses implement the list, we can instantiate a list object with any of these classes. For example,

```
List <T> al = new ArrayList<> ();
```

```
List <T> ll = new LinkedList<> ();
```

List <T> v = new Vector<> ();
Where T is the type of the object

4.ArrayList: ArrayList provides us with dynamic arrays in Java. Though, it may be slower than standard arrays but can be helpful in programs where lots of manipulation in the array is needed. The size of an ArrayList is increased automatically if the collection grows or shrinks if the objects are removed from the collection. Java ArrayList allows us to randomly access the list. ArrayList can not be used for primitive types, like int, char, etc.

5.LinkedList: LinkedList class is an implementation of the LinkedList data structure which is a linear data structure where the elements are not stored in contiguous locations and every element is a separate object with a data part and address part. The elements are linked using pointers and addresses. Each element is known as a node.

6.Vector: A vector provides us with dynamic arrays in Java. Though, it may be slower than standard arrays but can be helpful in programs where lots of manipulation in the array is needed. This is identical to ArrayList in terms of implementation. However, the primary difference between a vector and an ArrayList is that a Vector is synchronized and an ArrayList is non-synchronized.

Stack: Stack class models and implements the Stack data structure. The class is based on the basic principle of *last-in-first-out*. In addition to the basic push and pop operations, the class provides three more functions of empty, search and peek. The class can also be referred to as the subclass of Vector.

Queue Interface: As the name suggests, a queue interface maintains the FIFO(First In First Out) order similar to a real-world queue line. This interface is dedicated to storing all the elements where the order of the elements matter. For example, whenever we try to book a ticket, the tickets are sold at the first come first serve basis. Therefore, the person whose request arrives first into the queue gets the ticket. There are various classes like PriorityQueue, Deque, ArrayDeque, etc. Since all these subclasses implement the queue, we can instantiate a queue object with any of these classes.

For example,

Queue<T> pq = new PriorityQueue<> ();
Queue<T> ad = new ArrayDeque<> ();
Where T is the type of the object.

Priority Queue: A PriorityQueue is used when the objects are supposed to be processed based on the priority. It is known that a queue follows the First-In-First-Out algorithm, but sometimes the elements of the queue are needed to be processed according to the priority and this class is used in these cases. The PriorityQueue is based on the priority heap. The elements of the priority queue are ordered according to the natural ordering, or by a Comparator provided at queue construction time, depending on which constructor is used.

Deque Interface: This is a very slight variation of the queue data structure. Deque, also known as a double-ended queue, is a data structure where we can add and remove the

elements from both the ends of the queue. This interface extends the queue interface. The class which implements this interface is ArrayDeque. Since this class implements the deque, we can instantiate a deque object with this class. For example,

```
Deque<T> ad = new ArrayDeque<>();
```

Where *T* is the type of the object.

ArrayDeque: ArrayDeque class which is implemented in the collection framework provides us with a way to apply resizable-array. This is a special kind of array that grows and allows users to add or remove an element from both sides of the queue. Array deques have no capacity restrictions and they grow as necessary to support usage.

Set Interface: A set is an unordered collection of objects in which duplicate values cannot be stored. This collection is used when we wish to avoid the duplication of the objects and wish to store only the unique objects. This set interface is implemented by various classes like HashSet, TreeSet, LinkedHashSet, etc. Since all the subclasses implement the set, we can instantiate a set object with any of these classes.

HashSet: The HashSet class is an inherent implementation of the hash table data structure. The objects that we insert into the HashSet do not guarantee to be inserted in the same order. The objects are inserted based on their hashcode. This class also allows the insertion of NULL elements.

LinkedHashSet: A LinkedHashSet is very similar to a HashSet. The difference is that this uses a doubly linked list to store the data and retains the ordering of the elements.

Sorted Set Interface: This interface is very similar to the set interface. The only difference is that this interface has extra methods that maintain the ordering of the elements. The sorted set interface extends the set interface and is used to handle the data which needs to be sorted. The class which implements this interface is TreeSet. Since this class implements the SortedSet, we can instantiate a SortedSet object with this class.

TreeSet: The TreeSet class uses a Tree for storage. The ordering of the elements is maintained by a set using their natural ordering whether or not an explicit comparator is provided. This must be consistent with equals if it is to correctly implement the Set interface. It can also be ordered by a Comparator provided at set creation time, depending on which constructor is used.

Map Interface: A map is a data structure which supports the key-value pair mapping for the data. This interface doesn't support duplicate keys because the same key cannot have multiple mappings. A map is useful if there is a data and we wish to perform operations on the basis of the key. This map interface is implemented by various classes like HashMap, TreeMap etc. Since all the subclasses implement the map, we can instantiate a map object with any of these classes. For example,

```
Map<T> hm = new HashMap<>();
```

```
Map<T> tm = new TreeMap<>();
```

Where *T* is the type of the object.

HashMap: HashMap provides the basic implementation of the Map interface of Java. It stores the data in (Key, Value) pairs. To access a value in a HashMap, we must know its key. HashMap uses a technique called Hashing. Hashing is a technique of converting a large

String to small String that represents the same String so that the indexing and search operations are faster. HashSet also uses HashMap internally.

23. Define java bean and list out the steps of creating a new bean.

JavaBeans is a portable, platform-independent model written in Java Programming Language. Its components are referred to as beans.

In simple terms, JavaBeans are **classes** which encapsulate several **objects** into a single object. It helps in accessing these object from multiple places. JavaBeans contains several elements like Constructors, Getter/Setter Methods and much more.

JavaBeans has several conventions that should be followed:

- Beans should have a default **constructor** (no arguments)
- Beans should provide getter and setter methods
 - A *getter method* is used to read the value of a readable property
 - To update the value, a *setter method* should be called
- Beans should implement *java.io.Serializable*, as it allows to save, store and restore the state of a JavaBean you are working on

JavaBean Properties

A JavaBean property can be accessed by the user of the object. The feature can be of any Java data type, containing the classes that you define. It may be of the following mode: *read*, *write*, *read-only*, or *write-only*. JavaBean features are accessed through two **methods**:

getEmployeeName ()

For example, if the employee name is firstName, the method name would be getFirstName() to read that employee name. This method is known as an **accessor**. Properties of getter methods are as follows:

1. Must be public in nature
2. Return-type should not be void
3. The getter method should be prefixed with the word *get*
4. It should not take any argument

2.setEmployeeName ()

For example, if the employee name is firstName, the method name would be setFirstName() to write that employee name. This method is known as a **mutator**. Properties of setter methods:

1. Must be public in nature
2. Return-type should be void
3. The setter method has to be prefixed with the word set
4. It should take some argument

Advantages of JavaBeans

The following list enumerates some of the benefits of JavaBeans:

Portable

JavaBeans components are built purely in Java, hence are fully portable to any platform that supports the [Java Run-Time Environment](#). All platform specifics, as well as support for JavaBeans, are implemented by the Java Virtual Machine.

Compact and Easy

JavaBeans components are simple to create and easy to use. This is an important focus sector of the JavaBeans architecture. It doesn't take much effort to write a simple Bean. Also, a bean is lightweight, so, it doesn't have to carry around a lot of inherited baggage to support the Beans environment.

Carries the Strengths of the Java Platform

JavaBeans is pretty compatible, there isn't any new complicated mechanism for registering components with the run-time system.

Disadvantages of JavaBeans

1. JavaBeans are mutable, hence lack the advantages offered by immutable objects.
2. JavaBeans will be in inconsistent state partway through its construction.

Example Program: Implementation of JavaBeans

```
// Java Program of JavaBean class

package geeks;

public class Student implements java.io.Serializable
{

    private int id;

    private String name;

    public Student()

    {
```

```
    }

    public void setId(int id)

    {

        this.id = id;

    }

    public int getId()

    {

        return id;

    }

    public void setName(String name)

    {

        this.name = name;

    }

    public String getName()

    {

        return name;

    }

}

// Java program to access JavaBean class
```



```
package geeks;

public class Test {

    public static void main(String args[])

    {

        Student s = new Student(); // object is created

        s.setName("GFG"); // setting value to the object

        System.out.println(s.getName());

    }

}
```

Output:

GFG

Components of JavaBeans

Components of Java Beans are nothing but the class that contains the definition of beans. These classes have specific design conventions. It consists of events, methods, persistence, properties. Non-GUI based and GUI based components are the two types of components. The classes are discussed below:

- **Properties:** This attribute consists of a label, font, color, size, display size, etc. It determines behavior, appearance, and the state of Bean.
- **Methods:** Methods in JavaBeans are similar to the normal Java methods. It does have any specific naming convention.
- **Events:** Events in JavaBeans is similar to Swing event handling.
- **Persistence:** It helps in storing the state of JavaBean.

- JavaBeans components are Java classes that can be easily reused and composed together into applications. Any Java class that follows certain design conventions is a JavaBeans component.
- JavaServer Pages technology directly supports using JavaBeans components with standard JSP language elements.

Writing a Simple Bean

In this section you will learn more about Beans and the BeanBox by

- Creating a simple Bean
- Compiling and saving the Bean into a Java Archive (JAR) file
- Loading the Bean into the ToolBox
- Dropping a Bean instance into the BeanBox
- Inspecting the Bean's properties, methods, and events
- Generating an introspection report

Steps to create it and view it in the BeanBox:

1. **Write the SimpleBean code.** Put it in a file named SimpleBean.java, in the directory of your choice. Here's the code:

```
import java.awt.*;
import java.io.Serializable;

public class SimpleBean extends Canvas
    implements Serializable
{
    //Constructor sets inherited properties
    public SimpleBean(){
        setSize(60,40);
        setBackground(Color.red);
    }
}
```

SimpleBean extends the `java.awt.Canvas` component. SimpleBean also implements the `java.io.Serializable` interface, a requirement for all Beans. SimpleBean sets the background color and component size.

2. **Make sure the CLASSPATH environment variable is set** to point to all needed .class (or .jar) files. Here are some URLs that will help you to set CLASSPATH correctly:
 - The Managing Source and Class Files lesson gives good advice on how and *when* to set your CLASSPATH.
 - The JDK Tool Reference Page provides complete CLASSPATH information for both Windows and Solaris platforms.
3. **Compile the Bean:**

```
javac SimpleBean.java
```

This produces the class file SimpleBean.class

4. **Create a manifest file.** Use your favorite text editor to create a file, we'll call it manifest.tmp, that contains the following text:

```
Name: SimpleBean.class  
Java-Bean: True
```

5. **Create the JAR file.** The JAR file will contain the manifest and the SimpleBean class file:

```
jar cfm SimpleBean.jar manifest.tmp SimpleBean.class
```

See the JAR Files trail, and the JDK JAR file documentation for complete information on JAR files.

6. **Load the JAR file** into the ToolBox. Select the File|LoadJar... menu item. This will bring up a file browser. Navigate to the SimpleBean.jar location and select it. SimpleBean will appear at the bottom of the ToolBox. (Note that when the BeanBox is started, all Beans in JAR files in the beans/jars directory are automatically loaded into the ToolBox).
7. **Drop a SimpleBean instance into the BeanBox.** Click on the word SimpleBean in the ToolBox. The cursor will change to a crosshair. Move the cursor to a spot within the BeanBox and click. SimpleBean will appear as a painted rectangle with a hatched border. This border means that SimpleBean is selected. The SimpleBean properties will appear in the Properties sheet.

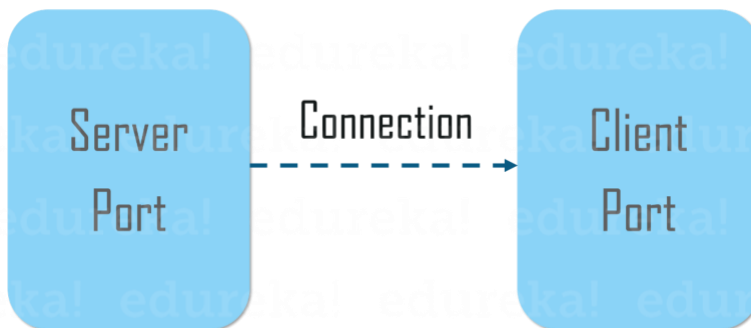
24. Explain the concept of socket with a suitable client/server code.

Socket Programming in Java

Socket programming is a way of connecting two nodes on a network to communicate with each other. One **socket** (node) listens on a particular port at an IP, while other *socket* reaches out to the other in order to form a connection.

Socket in Java

A **socket** in [Java](#) is one endpoint of a two-way communication link between two programs running on the network. A **socket** is bound to a port number so that the TCP layer can identify the application that data is destined to be sent to. A socket is one end-point of a two-way communication link between two programs running on the network.



An endpoint is a combination of an IP address and a port number. The package in the Java platform provides a class, `Socket` that implements one side of a two-way connection between your Java program and another program on the network. The class sits on top of a platform-dependent implementation, hiding the details of any particular system from your Java program. By using the class instead of relying on native code, your [Java programs](#) can communicate over the network in a platform-independent fashion.

Client Side Programming

Establish a Socket Connection

To connect to other machine we need a socket connection. A socket connection means the two machines have information about each other's network location (IP Address) and TCP port. The `java.net.Socket` class represents a `Socket`. To open a socket:

```
Socket socket = new Socket("127.0.0.1", 5000)
```

- First argument – **IP address of Server**. (127.0.0.1 is the IP address of localhost, where code will run on single stand-alone machine).
- Second argument – **TCP Port**. (Just a number representing which application to run on a server. For example, HTTP runs on port 80. Port number can be from 0 to 65535)

Communication

To communicate over a socket connection, streams are used to both input and output the data.

Closing the connection

The socket connection is closed explicitly once the message to server is sent.

Server Programming

Establish a Socket Connection

To write a server application two sockets are needed.

- A ServerSocket which waits for the client requests (when a client makes a new Socket())
- A plain old Socket socket to use for communication with the client.

Communication

getOutputStream() method is used to send the output through the socket.

Close the Connection

After finishing, it is important to close the connection by closing the socket as well as input/output streams.

MyServer.java

```
import java.io.*;
import java.net.*;
public class MyServer {
    public static void main(String[] args){
        try{
            ServerSocket ss=new ServerSocket(6666);
            Socket s=ss.accept();//establishes connection
            DataInputStream dis=new DataInputStream(s.getInputStream());
            String str=(String)dis.readUTF();
            System.out.println("message= "+str);
            ss.close();
        }catch(Exception e){System.out.println(e);}
    }
}
```

MyClient.java

```
import java.io.*;
import java.net.*;
public class MyClient {
    public static void main(String[] args) {
        try{
```

```
Socket s=new Socket("localhost",6666);
DataOutputStream dout=new DataOutputStream(s.getOutputStream());
dout.writeUTF("Hello Server");
dout.flush();
dout.close();
s.close();
    }catch(Exception e){System.out.println(e);}
}
```



The primary purpose of beans is to enable the visual construction of applications. You've probably used or seen applications like Visual Basic, Visual Age, or Delphi. These tools are referred to as visual **application builders**, or **builder tools** for short.

Typically such tools are GUI applications, although they need not be. There is usually a palette of components available from which a program designer can drag items and place them on a *form* or *client window*.

25. Explain the Application Builder Tools and JAR Files.

Builder tools allow a developer to work with JavaBeans in a convenient way. By examining a JavaBean by a process known as Introspection, a builder tool exposes the discovered features of the JavaBean for visual manipulation. A builder tool maintains a list of all JavaBeans available. It allows you to compose the Bean into applets, application, servlets and composite components (e.g. a JFrame), customize its behavior and appearance by modifying its properties and connect other components to the event of the Bean or vice versa.

When working with Java Beans, most developers use an application builder tool, a utility that enables. you to configure a set of Beans, connect them together, and produce a working application. Its major capabilities arc the following:

1. A palette is provided that lists all of the available Beans. As addition Beans are developed or purchased, they can be added to the palette.

2. A worksheet is displayed that allows the designer to lay out Beans in a graphical user interface. A designer may drag and drop a Bean from the palette to this worksheet.
3. Special editors and customizers allow a Bean to be configured. This is the mechanism by which the behavior of a Bean may be set in a particular environment.
4. Commands allow a designer to inquire about the state and behavior of a Bean. This information automatically becomes available when a Bean is added to the palette.
5. Capabilities exist to interconnect Beans. This means that events generated by one component are mapped to method invocations on other components.
7. When a collection of Beans has been configured and connected, it is possible to save all of this information in a persistent storage area. At a later time, this information can then be used to restore the state of the application.

JAR Files

A JAR (Java Archive) is a package file format typically used to aggregate many Java class files and associated metadata and resources (text, images, etc.) into one file to distribute application software or libraries on the Java platform. In simple words, a JAR file is a file that contains a compressed version of .class files, audio files, image files or directories. We can imagine a .jar file as a zipped file(.zip) that is created by using WinZip software. Even, WinZip software can be used to extract the contents of a .jar. So you can use them for tasks such as lossless data compression, archiving, decompression, and archive unpacking.

Let us see how to create a .jar file and related commands which help us to work with .jar files

1. **Create a JAR file:** To create a .jar file, we can use the `jar cf` command in the following way:

```
jar cf jarfilename inputfiles
```

Here, `cf` represents create the file. For example, assuming our package `pack` is available in `C:\directory`, to convert it into a jar file into the `pack.jar`, we can give the command as:

```
C:\> jar cf pack.jar pack
```

Now, `pack.jar` file is created

2. **Viewing a JAR file:** To view the contents of .jar files, we can use the command as:
`jar tf jarfilename`

Here, `tf` represents table view of file contents. For example, to view the contents of our `pack.jar` file, we can give the command:

```
C:/> jar tf pack.jar
```

Now, the contents of `pack.jar` are displayed as:

```
META-INF/
```

```
META-INF/MANIFEST.MF
```

```
pack/
```

```
pack/class1.class
```

```
pack/class2.class
```

```
..
```

```
..
```

where class1 , class2 etc are the classes in the package pack. The first two entries represent that there is a manifest file created and added to pack.jar. The third entry represents the sub-directory with the name pack and the last two represent the files name in the directory pack.

When we create .jar files , it automatically receives the default manifest file. There can be only one manifest file in an archive , and it always has the pathname.

META-INF/MANIFEST.MF

This manifest file is useful to specify the information about other files which are packaged.

3. **Extracting a JAR file:** To extract the files from a .jar file , we can use:

```
jar xf jarfilename
```

Here, xf represents extract files from the jar files. For example , to extract the contents of our pack.jar file, we can write:

```
C:\> jar xf pack.jar
```

This will create the following directories in C:\

META-INF

pack // in this directory , we can see class1.class and class2.class.

4. **Updating a JAR File** The Jar tool provides a 'u' option which you can use to update the contents of an existing JAR file by modifying its manifest or by adding files. The basic command for adding files has this format:

```
jar uf jar-file input-file(s)
```

here uf represent update jar file. For example , to update the contents of our pack.jar file, we can write:

```
C:\>jar uf pack.jar
```

5. **Running a JAR file:** In order to run an application packaged as a JAR file (requires the Main-class manifest header) , following command can be used:

```
C:\>java -jar pack.jar
```